

LAB 4 – VIBE CODING

Rhythm Tap Game – Vibe Coding Work Log / Chat History

Student	Tejas U L
SRN	PES1UG24CS715
Repository	TejasUL/51_rhythm-tap
Branch	Computer Science Engineering

1. Objective

- Use Vibe Coding to inspect, fix broken code, and add the features specified for the Rhythm Tap game.
- The work was performed on the student's personal fork of the assigned repository.

2. Repository

- Original repository: https://github.com/SETAPESU26/51_rhythm-tap
- Personal fork: https://github.com/TejasUL/51_rhythm-tap
- The final code was committed and pushed to the personal fork.

3. Initial Setup and Testing

- The project was run using Python 3.12 because the original environment had a pygame installation issue under a newer Python version.
- pygame 2.5.2 was installed successfully using the project's requirements.
- The original game was run and a 10-second BEFORE video was recorded before making the requested changes.

4. Initial Code Fix

- The game code was inspected to identify broken or incorrect behavior.
- A drawing issue was found where missed notes could still be drawn because the draw loop skipped hit notes but not missed notes.
- The draw condition was corrected so notes marked as hit or missed are not rendered.

5. Feature 1 – Sound Effects

- Sound effects were added for PERFECT, GREAT, and OK judgments.
- The assets created were: assets/perfect.wav, assets/great.wav, and assets/ok.wav.
- The game engine was updated to load these sounds and play the corresponding sound when a normal note receives a grade.
- A separate pygame sound test was performed to confirm that the audio system and WAV assets worked correctly.
- The game was then run successfully with the sound effects.

6. Feature 2 – Hold Notes

- Hold notes were added to the game.
- A hold note requires the player to press the correct lane key and keep holding it for 1 second.
- The note model was extended with hold-note state, hold duration, and hold/progress information.

- The game engine was updated to detect hold notes, track the key while it is held, fail the hold if the key is released early, and complete the hold after the required duration.
- A visual tail/progress indicator was added for hold notes.

7. Feature 3 – BPM-Synchronized Spawning

- The original frame-based spawning was replaced with BPM-based beat scheduling.
- The implementation uses $\text{BPM} = 120$ initially, with the beat interval calculated as $60000 / \text{BPM}$.
- Beat scheduling was implemented using a next-beat timestamp and advancing the scheduled timestamp by one beat interval to avoid timing drift.
- The feature was tested successfully.

8. Gameplay Readability Improvement

- During testing, normal notes and hold notes were found to be too similar visually, making them difficult to distinguish.
- The gameplay was also considered too fast for comfortable testing.
- The BPM was changed to 80, giving approximately 0.75 seconds between beats.
- The hold-note probability was reduced to 20% so hold notes would not appear too frequently.
- Hold notes were made visually distinct with a long outlined tail and a clearly visible HOLD label.

9. Feature 4 – Grade Summary

- A grade summary was added to the game-over screen.
- The summary displays PERFECT count, GREAT count, OK count, MISS count, and Accuracy percentage.
- Accuracy is calculated from successful judgments divided by the total recorded judgments.
- The R key can be used to restart the game after game over.
- The completed version was tested and confirmed working.

10. Final Testing

- A second 10-second AFTER video was recorded after completing the changes.
- The final game was tested with normal notes, hold notes, sound effects, BPM-based spawning, grading, misses, game over, accuracy display, and restart behavior.

11. GitHub Submission

- The local project was connected to the personal fork using the origin remote.
- Unnecessary generated files and temporary ZIP/copy files were removed before committing.
- The changes were staged with `git add`.
- Commit message used: Complete Lab 4 Vibe Coding features
- The commit was pushed successfully to the personal GitHub fork on the main branch.

12. Final Deliverables

- BEFORE video – completed.
- AFTER video – completed.
- Updated source code and sound assets – pushed to the personal GitHub fork.
- Vibe Coding work log / chat-history document – this document.

Note: This is a clean lab work log reconstructed from the Vibe Coding work completed in this chat. It summarizes the prompts, fixes, feature changes, testing, and GitHub steps rather than presenting itself as a verbatim export of every chat message.